

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

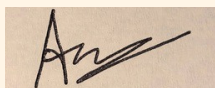
ASSESSMENT TOOL 1 – High (Coding Challenge)

Course Code:	CSA0901	Course Title:	Programming in Java
CO Assessed:		Assessment:	
Weightage:	40% of CIA	Total Marks:	100
CO1 Statement:	<i>Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)</i>		
Student Name:	S.Aarathana	Reg. No.:	1921260021
Section / Batch:	2026	Date:	22-07-2026

Code of Conduct

I S.Aarathana (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



Instructions

- This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 90 minutes.

Coding Challenge**1. Library Book Management using Classes & Encapsulation**

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

Aim

To develop a Library Book Management System using Java classes and encapsulation by creating a Book class with private data members, constructors, getters, setters, and methods to add, search, issue, and return books through a menu-driven program using an array of objects.

ALGORITHM**Step 1: Start****Step 2:**

Create a Book class with private data members:

- bookId
- title
- author
- price
- issued

Step 3:

Create

-
- Constructor
- Getters and Setters
- Methods:
 - issueBook()
 - returnBook()
 - display()

Step 4:

In the main program,

- Create an array of Book objects.
- Display a menu repeatedly.

Step 5:

Perform operations based on user choice:

- Add Book
- Display Books
- Search Book
- Issue Book
- Return Book
- Exit

Step 6: Stop

Code:

```
import java.util.Scanner;

class Book {
    private int id;
    private String title;
    private boolean issued;

    Book(int id, String title) {
        this.id = id;
        this.title = title;
        issued = false;
    }

    int getId() {
        return id;
    }

    void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        } else {
            System.out.println("Already Issued");
        }
    }

    void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned");
        } else {
            System.out.println("Book is Already Available");
        }
    }

    void display() {
        System.out.println(id + " " + title + " " + (issued ? "Issued" : "Available"));
    }
}
```

```
public class Library {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Book[] books = new Book[10];
        int count = 0, choice, id;

        do {
            System.out.println("\n1.Add 2.Display 3.Issue 4.Return 5.Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Book ID: ");
                    id = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Book Title: ");
                    String title = sc.nextLine();
                    books[count++] = new Book(id, title);
                    break;

                case 2:
                    for (int i = 0; i < count; i++)
                        books[i].display();
                    break;

                case 3:
                    System.out.print("Enter Book ID: ");
                    id = sc.nextInt();
                    for (int i = 0; i < count; i++)
                        if (books[i].getId() == id)
                            books[i].issueBook();
                    break;

                case 4:
                    System.out.print("Enter Book ID: ");
                    id = sc.nextInt();
                    for (int i = 0; i < count; i++)
                        if (books[i].getId() == id)
                            books[i].returnBook();
                    break;

                case 5:
                    System.out.println("Thank You");
                    break;

                default:
                    System.out.println("Invalid Choice");
            }
        } while (choice != 5);

        sc.close();
    }
}
```

Sample Input

1

101

Java Programming

1

102

Python Basics

2

3

101

4

101

2

5

Sample Output

1.Add 2.Display 3.Issue 4.Return 5.Exit

Enter Choice: 1

Book ID: 101

Book Title: Java Programming

1.Add 2.Display 3.Issue 4.Return 5.Exit

Enter Choice: 1

Book ID: 102

Book Title: Python Basics

1.Add 2.Display 3.Issue 4.Return 5.Exit

Enter Choice: 2

101 Java Programming Available

102 Python Basics Available

1.Add 2.Display 3.Issue 4.Return 5.Exit

Enter Choice: 3

Enter Book ID: 101

Book Issued

1.Add 2.Display 3.Issue 4.Return 5.Exit

Enter Choice: 4

Enter Book ID: 101

Book Returned

1.Add 2.Display 3.Issue 4.Return 5.Exit

Enter Choice: 5

Thank You

Coding Challenge**2. Employee Payroll System using Inheritance**

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

Aim

To develop an Employee Payroll System using inheritance and runtime polymorphism in Java by creating a base class Employee and derived classes PermanentEmployee and ContractEmployee to calculate and display employee salaries.

Algorithm

Step 1: Start**Step 2:**

Create a base class Employee with employee ID and name

Step 3:

1.Create subclasses PermanentEmployee and ContractEmployee.

Step 4:

Override the calculateSalary() method in each subclass.

Step 5:

Accept employee details from the user.

Step 6:

Use runtime polymorphism to call the appropriate calculateSalary() method.

Step 7:

Display employee details and salary.

Step 8: Stop

Code:

```
import java.util.Scanner;

class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void calculateSalary() {
        System.out.println("Salary");
    }
}

class PermanentEmployee extends Employee {
    double basic;

    PermanentEmployee(int id, String name, double basic) {
        super(id, name);
        this.basic = basic;
    }

    void calculateSalary() {
        System.out.println("Salary = " + (basic + basic * 0.20));
    }
}

class ContractEmployee extends Employee {
    int hours;
    double rate;

    ContractEmployee(int id, String name, int hours, double rate) {
        super(id, name);
        this.hours = hours;
        this.rate = rate;
    }

    void calculateSalary() {
        System.out.println("Salary = " + (hours * rate));
    }
}
```

```
public class Payroll {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Employee e;

        System.out.println("1.Permanent 2.Contract");
        int ch = sc.nextInt();

        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        if (ch == 1) {
            System.out.print("Enter Basic Salary: ");
            double basic = sc.nextDouble();
            e = new PermanentEmployee(id, name, basic);
        } else {
            System.out.print("Enter Hours Worked: ");
            int h = sc.nextInt();
            System.out.print("Enter Rate per Hour: ");
            double r = sc.nextDouble();
            e = new ContractEmployee(id, name, h, r);
        }

        System.out.println("\nEmployee ID: " + id);
        System.out.println("Employee Name: " + name);
        e.calculateSalary();

        sc.close();
    }
}
```

Sample Input (Permanent Employee)

1
101
Rahul
30000

Sample Output

1. Permanent 2. Contract

Enter ID: 101

Enter Name: Rahul

Enter Basic Salary: 30000

Employee ID: 101

Employee Name: Rahul

Salary = 36000.0

Sample Input (Contract Employee)

2
102
Priya
25
500

Sample Output

1. Permanent 2. Contract

Enter ID: 102

Enter Name: Priya

Enter Hours Worked: 25

Enter Rate per Hour: 500

Employee ID: 102

Employee Name: Priya

Salary = 12500.0

Coding Challenge**3. Vehicle Rental System using Runtime Polymorphism**

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

Aim

To develop a Vehicle Rental System using runtime polymorphism in Java by creating a superclass Vehicle and subclasses Car, Bike, and Bus to calculate rental charges dynamically.

Algorithm

Step 1: Start**Step 2:**

Create a superclass Vehicle with method calculateRent(int days).

Step 3:

Create subclasses Car, Bike, and Bus.

Step 4:

Override calculateRent() in each subclass.

Step 5:

Store objects in an array of Vehicle references.

Step 6:

Accept the number of rental days.

Step 7:

Display the rental bill using runtime polymorphism.

Step 8: Stop

Code:

```
import java.util.Scanner;

class Vehicle {
    void calculateRent(int days) {
    }
}

class Car extends Vehicle {
    void calculateRent(int days) {
        System.out.println("Car Rent = Rs." + (days * 1000));
    }
}

class Bike extends Vehicle {
    void calculateRent(int days) {
        System.out.println("Bike Rent = Rs." + (days * 300));
    }
}

class Bus extends Vehicle {
    void calculateRent(int days) {
        System.out.println("Bus Rent = Rs." + (days * 2000));
    }
}

public class VehicleRental {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Vehicle[] v = new Vehicle[3];
        v[0] = new Car();
        v[1] = new Bike();
        v[2] = new Bus();

        System.out.println("1.Car 2.Bike 3.Bus");
        System.out.print("Choose Vehicle: ");
        int ch = sc.nextInt();

        System.out.print("Enter Rental Days: ");
        int days = sc.nextInt();

        v[ch - 1].calculateRent(days);

        sc.close();
    }
}
```

Sample Input 1

1

5

Sample Output

1.Car 2.Bike 3.Bus

Choose Vehicle: 1

Enter Rental Days: 5

Car Rent = Rs.5000

Sample Input 2

2

4

Sample Output

1.Car 2.Bike 3.Bus

Choose Vehicle: 2

Enter Rental Days: 4

Bike Rent = Rs.1200

Sample Input 3

3

2

Sample Output

1.Car 2.Bike 3.Bus

Choose Vehicle: 3

Enter Rental Days: 2

Bus Rent = Rs.4000

Coding Challenge**4. Bank Account System using Data Hiding**

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

Aim

To develop a Bank Account System using data hiding (encapsulation) in Java by keeping account details private and providing methods for deposit, withdrawal, and balance inquiry with proper validation.

Algorithm

Step 1: Start**Step 2:**

1. Create a BankAccount class with private variables:
 - accountNumber
 - holderName
 - balance

Step 3:

Create a constructor to initialize the account details.

Step 4:

Implement methods:

- deposit()
- withdraw()
- displayBalance()

Step 5:

In the main method, create a BankAccount object.

Step 6:

Accept deposit and withdrawal amounts.

Step 7:

Display the updated balance.

Step 8: Stop

Code:

```
import java.util.Scanner;

class BankAccount {
    private int accountNumber;
    private String holderName;
    private double balance;

    BankAccount(int accNo, String name, double bal) {
        accountNumber = accNo;
        holderName = name;
        balance = bal;
    }

    void deposit(double amt) {
        if (amt > 0) {
            balance += amt;
            System.out.println("Amount Deposited");
        } else
            System.out.println("Invalid Amount");
    }

    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            System.out.println("Amount Withdrawn");
        } else
            System.out.println("Insufficient Balance");
    }

    void displayBalance() {
        System.out.println("Account No: " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance: " + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Account Number: ");
        int acc = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Holder Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Initial Balance: ");
        double bal = sc.nextDouble();

        BankAccount b = new BankAccount(acc, name, bal);

        System.out.print("Enter Deposit Amount: ");
        b.deposit(sc.nextDouble());

        System.out.print("Enter Withdrawal Amount: ");
        b.withdraw(sc.nextDouble());

        b.displayBalance();

        sc.close();
    }
}
```

Sample Input

101
Rahul
5000
2000
1500

Sample Output

Enter Account Number: 101
Enter Holder Name: Rahul
Enter Initial Balance: 5000
Enter Deposit Amount: 2000
Amount Deposited
Enter Withdrawal Amount: 1500
Amount Withdrawn
Account No: 101
Holder Name: Rahul
Balance: 5500.0

Sample Input 2

2

4

Sample Output

1.Car 2.Bike 3.Bus

Choose Vehicle: 2

Enter Rental Days: 4

Bike Rent = Rs.1200

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

Aim

To develop a Java program that demonstrates Method Overloading by calculating admission fees for Undergraduate, Postgraduate, and Scholarship students using overloaded calculateFee() methods.

Algorithm

1.START THE PROGRAM.

2.CREATE A CLASS NAMED ADMISSION.

3.DEFINE THREE OVERLOADED METHODS:

- calculateFee() – for Undergraduate students.
- calculateFee(int fee) – for Postgraduate students.
- calculateFee(int fee, int scholarship) – for Scholarship students.

4.DISPLAY A MENU.

5.READ THE USER'S CHOICE.

6.CALL THE CORRESPONDING OVERLOADED METHOD.

7.DISPLAY THE CALCULATED FEE.

8.STOP THE PROGRAM.

Code:

```
import java.util.Scanner;

class Admission
{
    void calculateFee()
    {
        System.out.println("Undergraduate Admission Fee = Rs.50000");
    }

    void calculateFee(int fee)
    {
        System.out.println("Postgraduate Admission Fee = Rs." + fee);
    }

    void calculateFee(int fee, int scholarship)
    {
        int finalFee = fee - scholarship;
        System.out.println("Scholarship Student Fee = Rs." + finalFee);
    }
}

public class Main
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        Admission obj = new Admission();

        System.out.println("1. Undergraduate");
        System.out.println("2. Postgraduate");
        System.out.println("3. Scholarship Student");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();

        switch(choice)
        {
            case 1:
                obj.calculateFee();
                break;

            case 2:
                System.out.print("Enter Postgraduate Fee: ");
                int pgFee = sc.nextInt();
                obj.calculateFee(pgFee);
                break;

            case 3:
                System.out.print("Enter Course Fee: ");
                int fee = sc.nextInt();

                System.out.print("Enter Scholarship Amount: ");
                int scholarship = sc.nextInt();

                obj.calculateFee(fee, scholarship);
                break;

            default:
                System.out.println("Invalid Choice");
        }

        sc.close();
    }
}
```

Sample Input 1

Enter your choice: 1

Sample Output

Undergraduate Admission Fee = Rs.50000

Sample Input 2

Undergraduate Admission Fee = Rs.50000

Sample Output

Postgraduate Admission Fee = Rs.70000

Sample Input 3

Enter your choice: 3

Enter Course Fee: 70000

Enter Scholarship Amount: 20000

Sample Output

Scholarship Student Fee = Rs.50000

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

Aim

To develop a Java program using an Abstract Class and Runtime Polymorphism to calculate the areas of Circle, Rectangle, and Triangle dynamically.

Algorithm

- 1.START THE PROGRAM.
- 2.CREATE AN ABSTRACT CLASS SHAPE WITH AN ABSTRACT METHOD AREA().
- 3.CREATE SUBCLASSES CIRCLE, RECTANGLE, AND TRIANGLE THAT OVERRIDE THE AREA() METHOD.
- 4.STORE THE SUBCLASS OBJECTS IN AN ARRAY OF SHAPE REFERENCES.
- 5.TRAVERSE THE ARRAY AND INVOKE THE AREA() METHOD.
- 6.DISPLAY THE AREA OF EACH SHAPE USING RUNTIME POLYMORPHISM.
- 7.STOP THE PROGRAM.

Code:

```
abstract class Shape
{
    abstract void area();
}

class Circle extends Shape
{
    double radius;

    Circle(double radius)
    {
        this.radius = radius;
    }

    void area()
    {
        double a = 3.14 * radius * radius;
        System.out.println("Area of Circle = " + a);
    }
}

class Rectangle extends Shape
{
    double length, breadth;

    Rectangle(double length, double breadth)
    {
        this.length = length;
        this.breadth = breadth;
    }

    void area()
    {
        double a = length * breadth;
        System.out.println("Area of Rectangle = " + a);
    }
}

class Triangle extends Shape
{
    double base, height;

    Triangle(double base, double height)
    {
        this.base = base;
        this.height = height;
    }

    void area()
    {
        double a = 0.5 * base * height;
        System.out.println("Area of Triangle = " + a);
    }
}

public class Main
{
    public static void main(String args[])
    {
        Shape s[] = new Shape[3];

        s[0] = new Circle(5);
        s[1] = new Rectangle(10, 5);
        s[2] = new Triangle(8, 6);

        for(int i = 0; i < s.length; i++)
        {
            s[i].area();
        }
    }
}
```

Sample Input

Circle Radius = 5

Rectangle Length = 10

Rectangle Breadth = 5

Triangle Base = 8

Triangle Height = 6

Sample Output

Area of Circle = 78.5

Area of Rectangle = 50.0

Area of Triangle = 24.0

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface `MedicalRecord` containing methods `addRecord()` and `displayRecord()`. Implement the interface in classes `Patient` and `Doctor`. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

Aim

To develop a Java program using an Interface to implement a Hospital Management System for Patient and Doctor records, demonstrating interface-based abstraction and object interaction.

Algorithm

- 1.START THE PROGRAM.
- 2.CREATE AN INTERFACE MEDICALRECORD WITH METHODS ADDRECORD() AND DISPLAYRECORD().
- 3.CREATE A CLASS PATIENT THAT IMPLEMENTS THE INTERFACE.
- 4.CREATE A CLASS DOCTOR THAT IMPLEMENTS THE INTERFACE.
- 5.ACCEPT THE DETAILS OF THE PATIENT AND DOCTOR.
- 6.DISPLAY THE ENTERED RECORDS.
- 7.STOP THE PROGRAM.

Code:

```
import java.util.Scanner;

interface MedicalRecord
{
    void addRecord();
    void displayRecord();
}

class Patient implements MedicalRecord
{
    String name;
    int age;

    Scanner sc = new Scanner(System.in);

    public void addRecord()
    {
        System.out.print("Enter Patient Name: ");
        name = sc.nextLine();

        System.out.print("Enter Patient Age: ");
        age = sc.nextInt();
        sc.nextLine();
    }

    public void displayRecord()
    {
        System.out.println("\nPatient Record");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}

class Doctor implements MedicalRecord
{
    String name;
    String specialization;

    Scanner sc = new Scanner(System.in);

    public void addRecord()
    {
        System.out.print("\nEnter Doctor Name: ");
        name = sc.nextLine();

        System.out.print("Enter Specialization: ");
        specialization = sc.nextLine();
    }

    public void displayRecord()
    {
        System.out.println("\nDoctor Record");
        System.out.println("Name      : " + name);
        System.out.println("Specialization : " + specialization);
    }
}

public class Main
{
    public static void main(String args[])
    {
        MedicalRecord patient = new Patient();
        MedicalRecord doctor = new Doctor();

        patient.addRecord();
        doctor.addRecord();

        System.out.println("\n----- Hospital Records -----");
        patient.displayRecord();
        doctor.displayRecord();
    }
}
```

Sample Input

Enter Patient Name: Rahul

Enter Patient Age: 22

Enter Doctor Name: Dr.

Meena

Enter Specialization:

Cardiology

Sample Output

----- Hospital Records -----

Patient Record

Name : Rahul

Age : 22

Doctor Record

Name : Dr. Meena

Specialization : Cardiology

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

Aim

To develop a Java program using Packages by creating separate packages for Product and Order, importing them into the main application to place orders and generate the total bill.

Algorithm

- 1.START THE PROGRAM.
- 2.CREATE THE PACKAGE SHOPPING.PRODUCT WITH THE PRODUCT CLASS.
- 3.CREATE THE PACKAGE SHOPPING.ORDER WITH THE ORDER CLASS.
- 4.IMPORT BOTH PACKAGES INTO THE MAIN APPLICATION.
- 5.ACCEPT THE PRODUCT DETAILS AND QUANTITY.
- 6.CALCULATE THE TOTAL AMOUNT.
- 7.DISPLAY THE ORDER DETAILS AND BILL.
- 8.STOP THE PROGRAM.

Code:

```
package shopping.product;
```

```
public class Product
{
    public String productName;
    public double price;

    public Product(String productName, double price)
    {
        this.productName = productName;
        this.price = price;
    }
}
```

```
package shopping.order;
```

```
import shopping.product.Product;
```

```
public class Order
{
    Product p;
    int quantity;

    public Order(Product p, int quantity)
    {
        this.p = p;
        this.quantity = quantity;
    }

    public double calculateTotal()
    {
        return p.price * quantity;
    }

    public void displayBill()
    {
        System.out.println("Product Name : " + p.productName);
        System.out.println("Price      : Rs." + p.price);
        System.out.println("Quantity   : " + quantity);
        System.out.println("Total Amount : Rs." + calculateTotal());
    }
}
```

```
import shopping.product.Product;
import shopping.order.Order;
```

```
public class Main
{
    public static void main(String args[])
    {
        Product p = new Product("Laptop", 50000);

        Order o = new Order(p, 2);

        o.displayBill();
    }
}
```

Sample Input

Product Name : Laptop

Price : 50000

Quantity : 2

Sample Output

Product Name : Laptop

Price : Rs.50000.0

Quantity : 2

Total Amount : Rs.100000.0

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

Aim

To develop a Java program using an array of objects to store multiple student records and calculate total marks, average, grade, and class topper.

Algorithm

- 1.START THE PROGRAM.
- 2.CREATE A STUDENT CLASS WITH ROLL NUMBER, NAME, AND MARKS IN THREE SUBJECTS.
- 3.DEFINE METHODS TO CALCULATE TOTAL, AVERAGE, AND GRADE.
- 4.CREATE AN ARRAY OF STUDENT OBJECTS.
- 5.READ STUDENT DETAILS.
- 6.DISPLAY EACH STUDENT'S TOTAL, AVERAGE, AND GRADE.
- 7.FIND THE STUDENT WITH THE HIGHEST TOTAL MARKS.
- 8.DISPLAY THE CLASS TOPPER.
- 9.STOP THE PROGRAM.

Code:

```
import java.util.Scanner;

class Student
{
    int rollNo;
    String name;
    int m1, m2, m3;
    int total;
    double average;

    void getData(Scanner sc)
    {
        System.out.print("Enter Roll No: ");
        rollNo = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Name: ");
        name = sc.nextLine();

        System.out.print("Enter Marks in 3 Subjects: ");
        m1 = sc.nextInt();
        m2 = sc.nextInt();
        m3 = sc.nextInt();
    }

    void calculate()
    {
        total = m1 + m2 + m3;
        average = total / 3.0;
    }

    String getGrade()
    {
        if(average >= 90)
            return "A";
        else if(average >= 75)
            return "B";
        else if(average >= 50)
            return "C";
        else
            return "Fail";
    }

    void display()
    {
        System.out.println("\nRoll No : " + rollNo);
        System.out.println("Name : " + name);
        System.out.println("Total : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade : " + getGrade());
    }
}

public class Main
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        Student s[] = new Student[3];

        for(int i = 0; i < s.length; i++)
        {
            s[i] = new Student();
            s[i].getData(sc);
            s[i].calculate();
        }

        int topper = 0;

        System.out.println("\n----- Student Results -----");

        for(int i = 0; i < s.length; i++)
        {
            s[i].display();

            if(s[i].total > s[topper].total)
                topper = i;
        }

        System.out.println("\nClass Topper");
        System.out.println("Name : " + s[topper].name);
        System.out.println("Total : " + s[topper].total);

        sc.close();
    }
}
```

Sample Input

Student 1
Roll No: 101
Name: Rahul
Marks: 90 85 88

Student 2
Roll No: 102
Name: Priya
Marks: 95 92 94

Student 3
Roll No: 103
Name: Arjun
Marks: 80 78 82

Sample Output

----- Student Results -----

Roll No : 101
Name : Rahul
Total : 263
Average : 87.67
Grade : B

Roll No : 102
Name : Priya
Total : 281
Average : 93.67
Grade : A

Roll No : 103
Name : Arjun
Total : 240
Average : 80.0
Grade : B

Class Topper
Name : Priya
Total : 281

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Aim

To develop a Java program using Hierarchical Inheritance and Runtime Polymorphism to control smart home devices such as Light, Fan, and AirConditioner, and display their power consumption.

Algorithm

- 1.START THE PROGRAM.
- 2.CREATE A BASE CLASS DEVICE WITH METHODS TURNON(), TURNOFF(), AND SHOWPOWER().
- 3.CREATE DERIVED CLASSES LIGHT, FAN, AND AIRCONDITIONER THAT OVERRIDE THESE METHODS.
- 4.CREATE AN ARRAY OF DEVICE REFERENCES.
- 5.STORE OBJECTS OF LIGHT, FAN, AND AIRCONDITIONER IN THE ARRAY.
- 6.USE A LOOP TO INVOKE METHODS THROUGH THE BASE CLASS REFERENCE.
- 7.DISPLAY THE DEVICE STATUS AND POWER CONSUMPTION.
- 8.STOP THE PROGRAM.

Code:

```
class Device
{

    void turnOn()
    {
        System.out.println("Device is ON");
    }

    void turnOff()
    {
        System.out.println("Device is OFF");
    }

    void showPower()
    {
        System.out.println("Power Consumption");
    }
}

class Light extends Device
{

    void turnOn()
    {
        System.out.println("Light is ON");
    }

    void turnOff()
    {
        System.out.println("Light is OFF");
    }

    void showPower()
    {
        System.out.println("Power Consumption: 20 Watts");
    }
}

class Fan extends Device
{

    void turnOn()
    {
        System.out.println("Fan is ON");
    }

    void turnOff()
    {
        System.out.println("Fan is OFF");
    }

    void showPower()
    {
        System.out.println("Power Consumption: 75 Watts");
    }
}

class AirConditioner extends Device
{

    void turnOn()
    {
        System.out.println("Air Conditioner is ON");
    }

    void turnOff()
    {
        System.out.println("Air Conditioner is OFF");
    }

    void showPower()
    {
        System.out.println("Power Consumption: 1500 Watts");
    }
}

public class Main
{

    public static void main(String args[])
    {
        Device devices[] = new Device[3];

        devices[0] = new Light();
        devices[1] = new Fan();
        devices[2] = new AirConditioner();

        for(int i = 0; i < devices.length; i++)
        {
            devices[i].turnOn();
            devices[i].showPower();
            devices[i].turnOff();
            System.out.println();
        }
    }
}
```

Sample Input 3

No user input required.

Sample Output

Light is ON

Power Consumption: 20 Watts

Light is OFF

Fan is ON

Power Consumption: 75 Watts

Fan is OFF

Air Conditioner is ON

Power Consumption: 1500 Watts

Air Conditioner is OFF

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks	
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints		
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach		
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results		
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints		
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases		

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____